

Allow named modules to export macros

Document #: P3686R0
Date: 2025-5-8
Project: Programming Language C++
Audience: Core Language Evolution Group
Reply-to: Chuanqi Xu
<chuanqi.xcq@alibaba-inc.com>

§ 1 Background

Named modules were not allowed to export macros. But in practice we found it will be pretty helpful to allow named modules to export macros. We implemented and maintained it in our downstream clang based compiler and our products have been using it for 3 years. I tried to send this as an extension to the clang community. (<https://discourse.llvm.org/t/rfc-extensions-to-export-macros-preprocessor-states-for-c-20-modules/85083>) People there suggests me to register a paper to WG21 as part of the process.

§ 2 Proposal

The author of named modules can use annotations to offer macros. And the user need to import macros from named modules explicitly.

The author can use the paired annotations to offer macros:

```
export module A;  
#pragma modules "export-macro-begin"  
#define VALUE 43  
#pragma modules "export-macro-end"
```

as a natural grammar sugar of the paired annotations, user can do the following as well:

```
export module A;  
#pragma modules "export-macro-all"  
#define VALUE 43
```

then users can try to get the exported macros:

```
import A [[try_import_macros]];  
static_assert(VALUE == 43); // OK
```

but if the user don't want macros, the user won't get it.

```
import A;  
static_assert(VALUE == 43); // FAIL
```

§ 3 Motivation

Exporting macro from named modules by itself is a nice feature that users can understand to use it.

We describe two concrete motivations here to make it more explicit.

§ 3.1 Motivation1

Exporting macros from named modules help compilers to avoid redeclarations between BMIs.

It is a known problem that, the compiler (at least, clang) can't run efficient enough if different BMIs contain the same declarations. If users use named modules with headers for thirdparty dependencies, in case they have a lot of 3rd party dependencies, they will find their compile time goes up after switching to modules.

Although we can blame this is a quality of implementation issue, but from the perspective of the compiler developers (at least clang), we don't know how to solve the problem fundamentally. We (as clang developers) always suggest users to build modules bottom up, no matter if they are using C++20 named modules or clang header modules extension.

With allowing named modules to export macros, we can solve the problem easily:

```
module;  
import thirdparty_modules;  
#include <my_headers.h>  
export module my_mod;  
...
```

Now we need to touch `my_headers.h` to if-def out all the inclusion to 3rd party headers to get the best performance. But with the extension, it will be much easier.

§ 3.2 Motivation2

When we convert a project from headers to module interfaces (not wrapping headers into a modules but using modules), if a header contains macros, we need to move the macro definition part to a separate file. This changes the

structure of the project. While refactoring is not bad, it adds the cost for people to use modules.

With this extension, we can try to convert a header-based project to module-based one by tools, e.g., <https://github.com/ChuanqiXu9/clang-modules-converter>, a one-to-one mapping model.

Note that this is not preventing people to do refactoring, but it makes it easier for people who want to use modules and people can refactor their projects after the conversion too.

§4 Build Systems

One concern for the proposal was, how can the build systems handle the following case?

```
// A.cppm
export module A;
#pragma modules "export-macro-begin"
#define A_DEF
#pragma modules "export-macro-end"
```

```
import A [[try_import_macros]];
#ifdef A_DEF
import B;
#else
import C;
#endif
```

Currently the main stream build system (cmake) assumes the scanning process can be fully paralleled. To fully support the above case, the build system will have to refactor their model.

But the proposal is not about build systems. In practice, we assumed the exported macros won't affect the dependency scanning (implemented by not importing macros from named modules in clang-scan-deps) and we do this actually for 3 years. No problem happens.

I'm not saying we should ignore this question, the build systems will always have the chance to refactor their model to support the case. But users can ignore it in certain cases, users know their needs.

§ 5 Header Units

Actually the proposal itself is not related to header units. It is a self-contained story. But in clang's discourse thread, the most discussion is about header units. Since the **ideally implemented** header units can solve the first motivation case.

The story is, the standard allows the implementation to replace an inclusion to an importable header to an import.

So **ideally**, for the first motivation case, the implementation can replace **all** inclusions to the corresponding imports so that there won't be duplicated declarations in different BMIs (in a compilation).

As an example,

```
// V.cppm
module;
#include <vector> // or import <vector>;
export module V;
...
```

```
// S.cppm
module;
#include <string> // or import <string>;
export module S;
...
```

```
// U.cpp
import V;
import S;
...
```

To make it efficient, it is not enough to build 2 BMIs for the headers (vector and string). Since there are duplicated parts in vector and string. The build system needs to build BMIs for every included headers.

But header units is not implemented in the ideal way, at least in clang and CMake. Now is 2025 and header units is added in 2019, but we still don't get there.

And although there were voices that header units are basically clang header modules, given clang header modules is implemented, header units are implemented. Despite there is no document or practice for header units with clang in any non-toy project today, I spent 3 hours to experience clang header modules recently, and the build time increase from 6min to 12min in the testing project. Maybe it comes I triggered some redeclarations somehow, but I can't call the current status as usable or implemented.

No matter what, header units or clang header modules are not the topic. The proposal itself is not exclusive with header units.

§6 Macro Redefines and Undefs

The principle for redefines and undefs of exporting macro is, only the last #define or #undef in export macro region works.

e.g.,

```
export module m;  
#pragma modules "export-macro-begin"  
#define A_DEF  
#undef A_DEF // A_DEF won't be exported.  
#pragma modules "export-macro-end"
```

```
export module m;  
#pragma modules "export-macro-begin"  
#define A_DEF // A_DEF will be exported  
#pragma modules "export-macro-end"  
#undef A_DEF
```

```
export module m;  
#pragma modules "export-macro-begin"  
#define A_DEF 43  
#pragma modules "export-macro-end"  
  
#pragma modules "export-macro-begin"  
#define A_DEF 44 // A_DEF will be exported as 44  
#pragma modules "export-macro-end"  
#undef A_DEF
```

```
export module m;  
#pragma modules "export-macro-begin"  
#define A_DEF 43 // A_DEF will be exported as 43  
#pragma modules "export-macro-end"  
  
#define A_DEF 44  
#undef A_DEF
```

§ 7 Summary

The proposal is implemented and integrated in a downstream of clang. We've maintained it for 3 years and our product have been using this feature for years. This helps us to use named modules in product successfully. In our maintaining experience, the feature is pretty stable and maintainable. We rarely see bugs from it and cherry-pick it between different versions is easy.

There were voices in the community complaining that adopting named modules is hard. We believe this proposal can help users to adopt named modules much faster.

The build systems still have the chance to modify their dependency management model to support this feature. But users can use this feature without the support from build systems too.

This feature is not related to header units.